

Open in app ↗

Sign up

Sign in

Medium

 Search Write

Neo4j Developer Blog

Codebase Knowledge Graph

.NET code analysis using Neo4j Database



Vlad Batushkov

Follow

9 min read · Jul 6, 2021



40



This article is an introduction into a field of graph-based code analysis. My name is Vlad, I am a Tech Lead at Agoda, and have spent the last four years working on architectural changes of high-load systems, including a clean architecture redesign of Gateway API and modularization of the Agoda website.

Today, we will discuss a base concept of graph-based code analysis and learn how to build a Codebase Knowledge Graph (or Code Knowledge Graph or simply CKG) for a .NET Core project using Strazh.

- **Codebase Knowledge Graph:** a graph concept agnostic to a programming language.
- **Strazh:** tool to build a Codebase Knowledge Graph from a C# codebase.

This article uses examples of C# codebases. But if you are not familiar with C#, this article will still be useful for you. The concept of the Codebase Knowledge Graph is universal; you can learn the concept and create your own implementation of a Graph Builder for your favorite language.

How it works

Introduction

When talking about C# code analysis, we always talking about the Roslyn compiler. Roslyn rules are a well-known practice to analyze the code and improve code quality.

Libraries like SonarAnalyzer.CSharp, Microsoft.VisualStudio.Threading.Analyzers, AsyncFixer, and Agoda's in-house library Agoda.Analyzer are the perfect tools to catch “code smells” in a codebase: don't use this — use that; avoid this — prefer that, and so on.

From my personal experience, I can say that the majority of developers are interested in code analysis and want to have this knowledge in their skills toolbox.

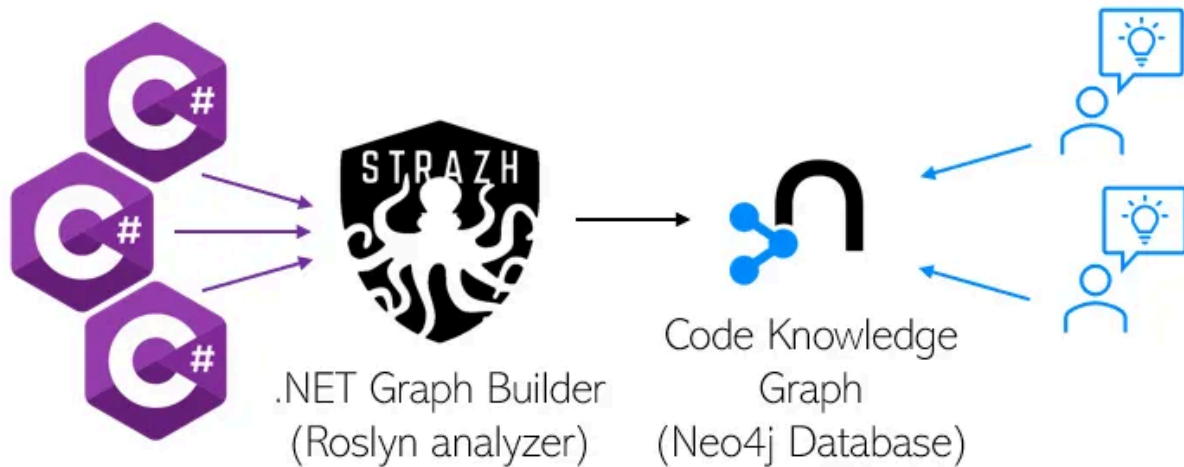
I decide to step forward in the code analysis and create an ability to analyze codebase in a different direction.

This is the idea:

Let's collect information about the project and build a graph from it.

Then, having a codebase as a graph, we can explore it, query the data, and analyze nodes and relationships in a graph as they are elements of a code.

Code analysis not used by writing Roslyn rules, but by using a Cypher queries.



Strazh literally means “a guardian.” This is a reference to Zion guardians, also known as Sentinel or “squiddy” from the original Matrix movie. (Other names in the Neo4j family — like Neo4j itself, Cypher, and APOC — are also taken from the Matrix universe.)

Example

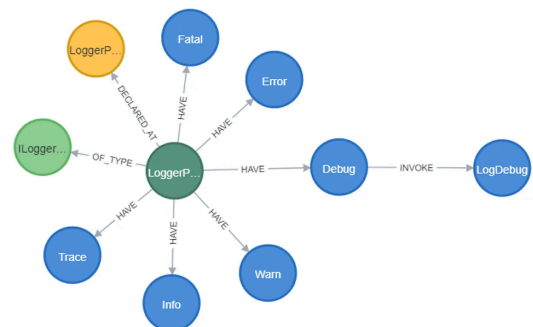
This is a simple class of *LoggerProviderAdapter* transformed into a graph structure:

```
public class LoggerProviderAdapter : ILoggerAdapter
{
    private readonly ILogger<LoggerProviderAdapter> _logger;

    public LoggerProviderAdapter(ILogger<LoggerProviderAdapter> logger)
    {
        _logger = logger;
    }

    public void Debug(string message, Exception exception = null, int pageType = 0)
    {
        _logger.LogDebug(message);
    }

    ...
}
```



You can see the graph of next code elements:

- **class** *LoggerProviderAdapter* (dark green node)
- **interface** *ILoggerAdapter* (light green node)
- the **file**, where this class is declared (yellow node)
- **methods** (blue nodes)

As you can see, *Debug* method called *LogDebug* method is not a method of *LoggerProviderAdapter*; otherwise it would also have a relationship to this class.

Why is it useful?

Graph-based analysis does not aim to replace Roslyn rules. Rather, it is an alternative technique that enriches our toolbox with exciting possibilities, such as:

- Fast access to various codebase insights with rich visualization (Neo4j Browser, Bloom, and other Neo4j tools)
- Ability to catch design violations and mistakes
- Ability to find different coverages and various statistical information
- Help you to estimate changes, features, and refactoring

Graph generation

Theory

First, I want to explain how graph generation works, and then we will discuss the graph itself. This section is a quick tour of how the transformation from codebase to graph technically works.

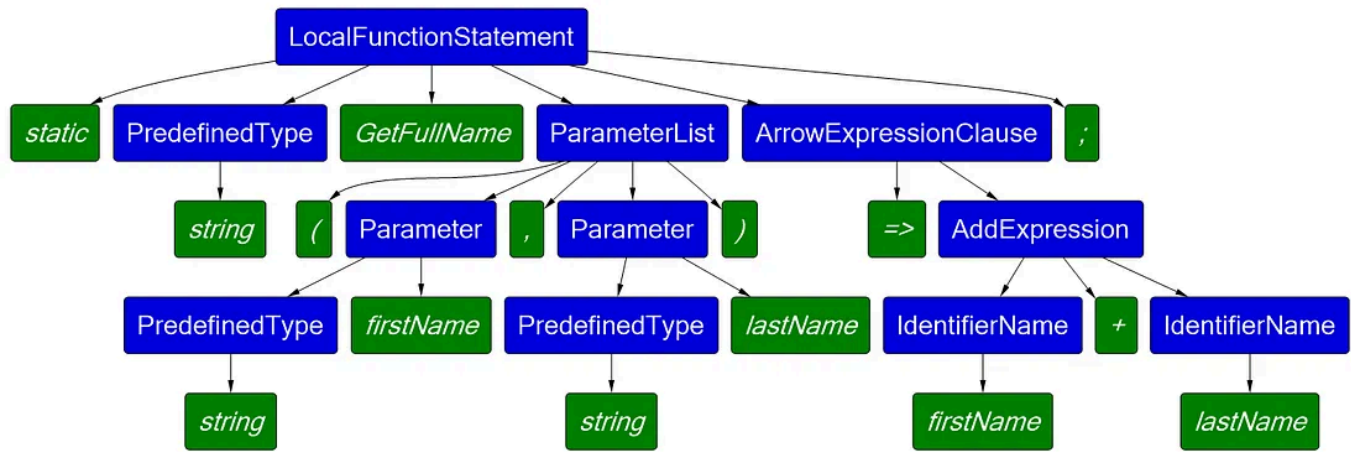
To generate a graph from C# codebase I've created a .NET Core console application Strazh, written also in C#. The application is also able to run as a Docker container. We will try Strazh in action at the end of this article.

From the technical point of view Strazh is an ETL tool (Extract, Transform, Load). Building a Codebase Knowledge Graph is the next process. First, we need to **Extract codebase models**, then we **Transform models into RDF triple**, and finally, we **Load RDF triple into a graph database** as the graph's nodes and relationships.

Syntax and Semantic Tree

Code of program is constructed from a combination of programming language syntax and semantics. Syntax is a grammar to define that combination of symbols is a valid code. Semantics gives a meaning to syntactic constructions — it is a layer of abstraction over acting code models.

```
static string GetFullName(string firstName, string lastName)
    => firstName + lastName;
```



Example of Syntax Tree (LINQPad)

Roslyn analyzer

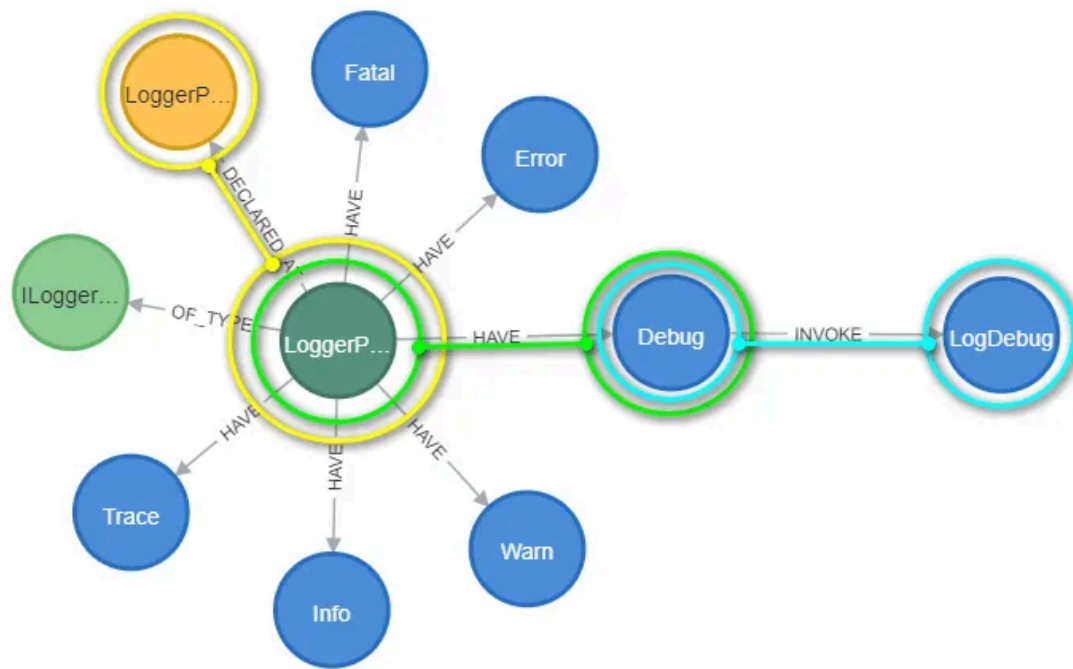
Roslyn is a powerful .NET compiler that provides APIs for code analysis. You can traverse in your codebase exploring Syntax and Semantic Trees to extract from it anything you want.

RDF triple

RDF (Resource Description Framework), or a semantic triple, is a specific structure that used to represent facts. RDF triple is based on expression: subject — predicate — object. Codebase semantic models are extracted from code and transformed into RDF triples. Now that we have a collection of RDF triples we can build a graph from it.

Back to *LoggerProviderAdapter*, here is a RDF triple example:

- Subject “class *LoggerProviderAdapter*” (Node :Class)
- Predicate “have” (Relationship :HAVE)
- Object “method *Debug*” (Node :Method)



Examples of Triples: Class declared in a File. Class have a Method. Method invoke some another Method.

This is what a graph builder is doing: compile source code of project, extract known semantic models as RDF triples, and insert them into database to build a graph.

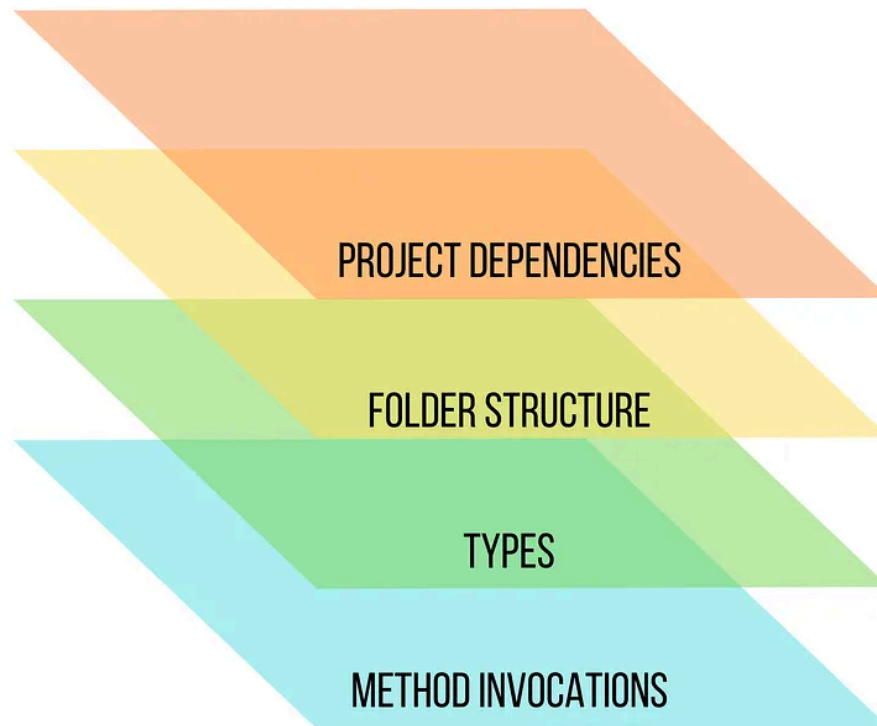
So far so good, and now we are going to take a deep-dive into details of CKG and explore graph schema.

Codebase Knowledge Graph

Graph basics

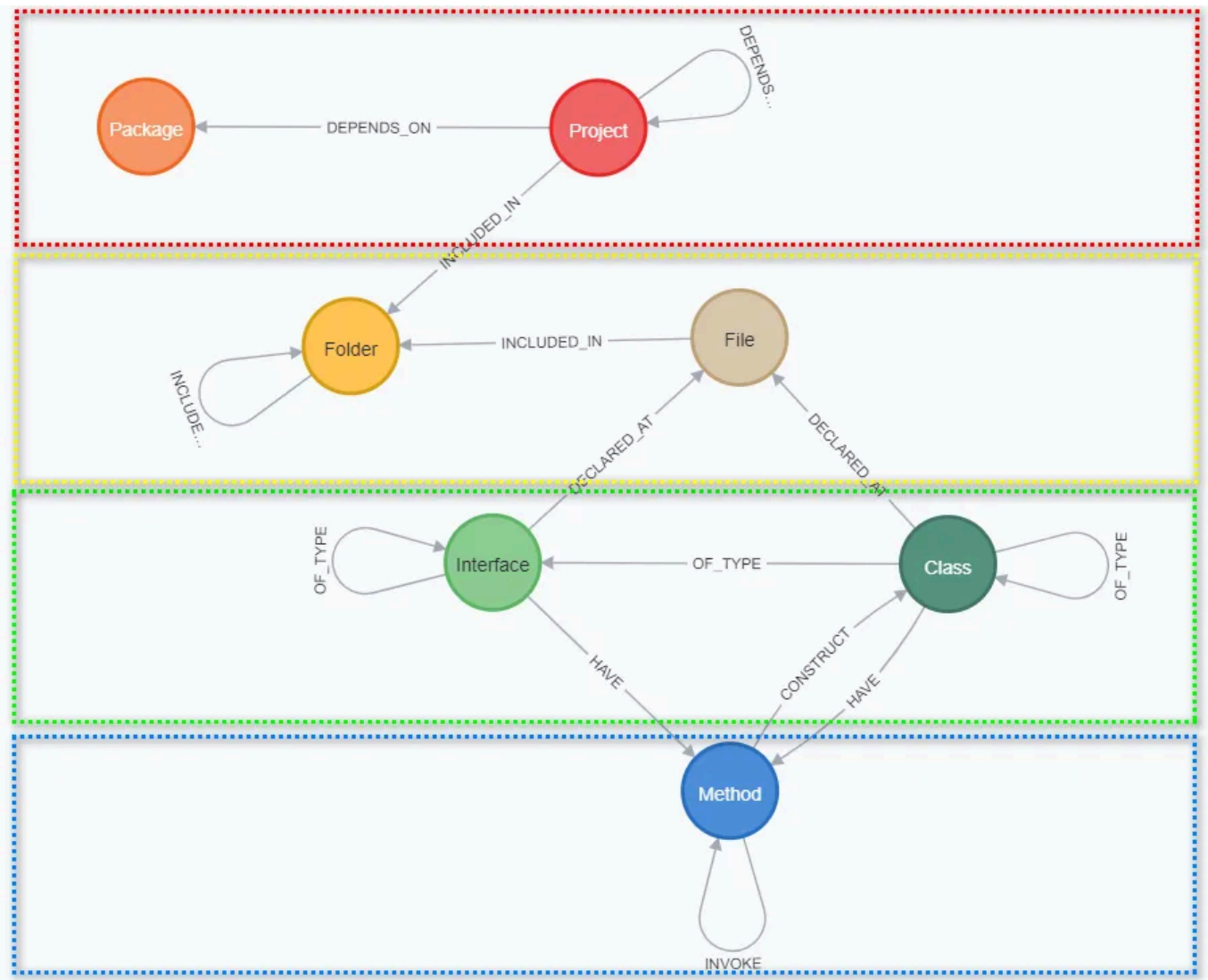
Knowledge Graph is a graph-structured model that integrates free-form data into a network of entities, their semantic types, properties, and relationships. Free-form is a very important statement here. The more data we use to build a graph, the more useful insights we can mine from it.

Codebase Knowledge Graph is a graph-structured dataset with free-form relationships between entities of codebase, semantic models of programming language, project structure, and other interlinked knowledges.



Codebase Knowledge Graph layers

All layers intercept with each other to give you a powerful set of analytical information of your codebase. Lets observe graph schema and explore layers one by one, from top to the bottom.

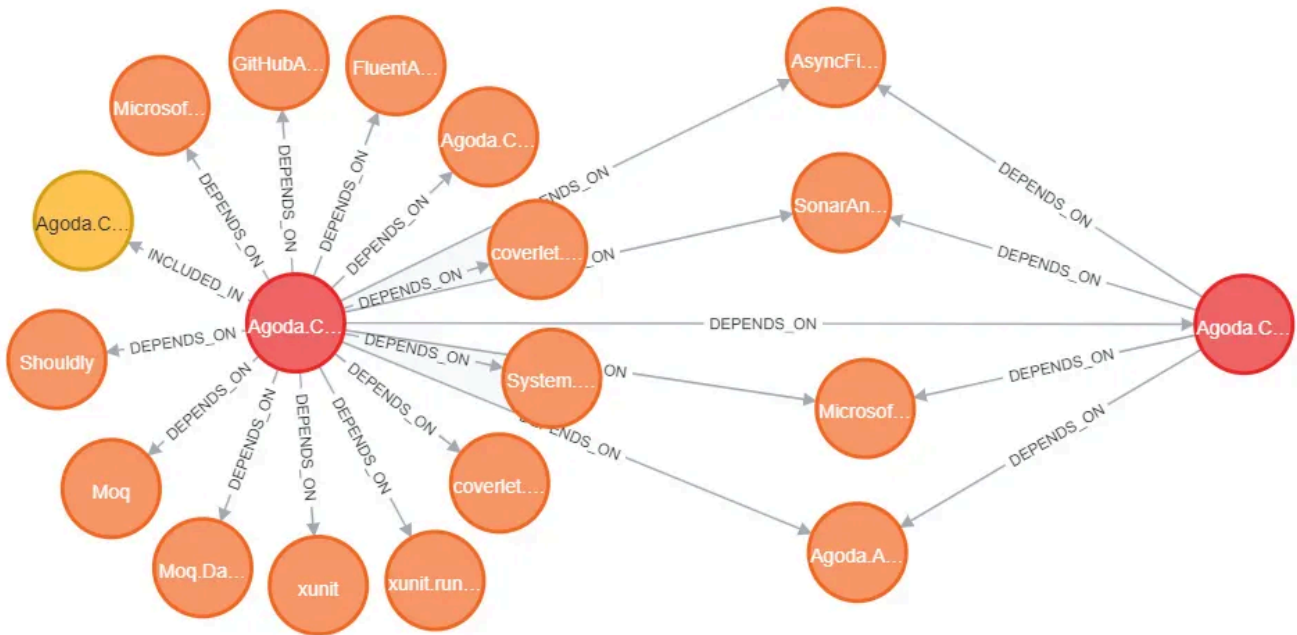


Codebase Knowledge Graph schema

Project Dependencies

On the top level we have **Projects** and **Packages**.

This layer is good for an architectural overview and to help with design decisions. Using this layer, you can explore dependencies between Projects. Primitive example below.



UnitTests project depends on Project (red nodes) with expanded Packages (orange nodes) used by UnitTests

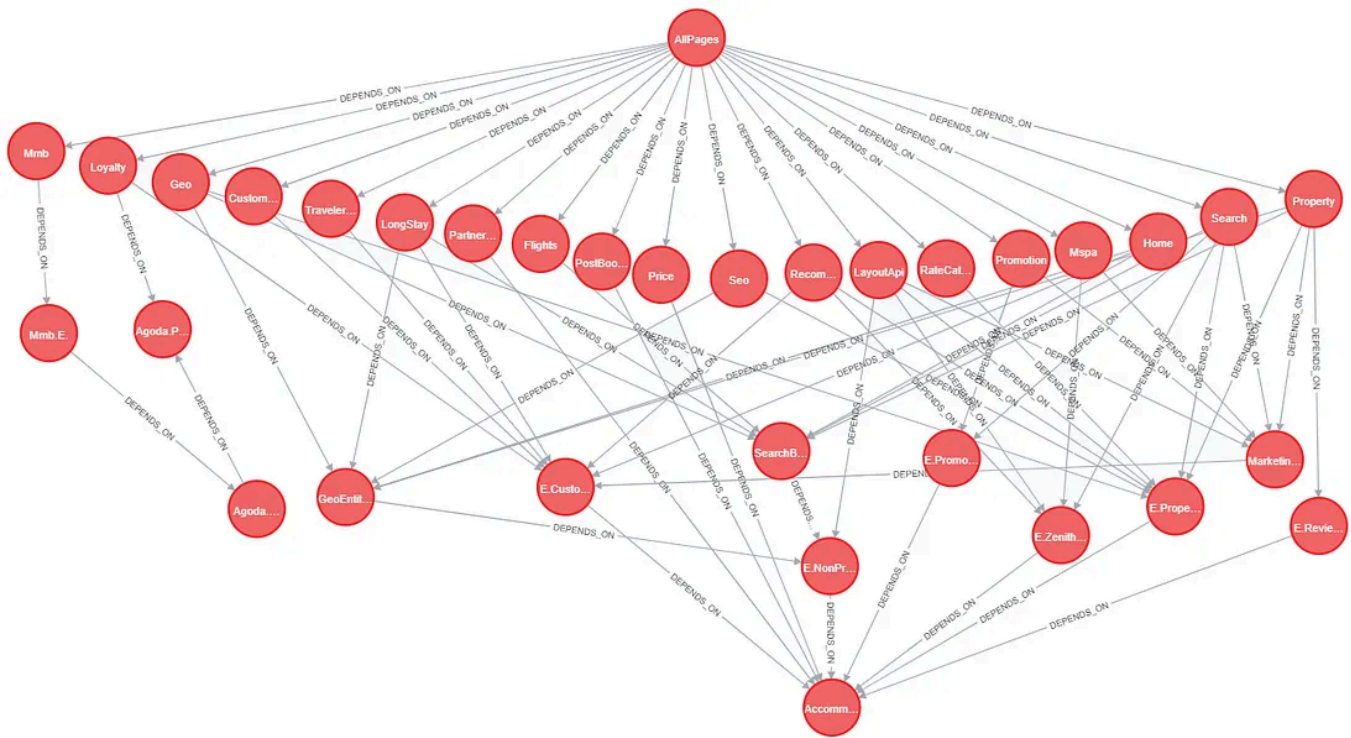
During a migration phase of our modular redesign of the Agoda website, at some point we had around 70 Projects in a single solution.

Breaking the Monolith

Modular redesign of Agoda.com

medium.com

Codebase Knowledge Graph helps us catch horrible dependency violations and fix them at an early stage.



Project Dependencies

Yes, there are some existing tools to get the same diagram, but Neo4j gives us a long list of options to explore solution and codebase layers: Cypher, APOC, DGS, Neo4j Browser, Bloom, and many more native tools are fast and easy to use.

Get Vlad Batushkov's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Folder Structure

Going one level deeper we have a File System layer to structure **Folders** and **Files**. This layer is useful data to validate correctness of a project's structure, when the project follows any standard pattern.

Pattern-Oriented Software Architecture - Wikipedia

Pattern-Oriented Software Architecture is a series of software engineering books describing software design patterns.

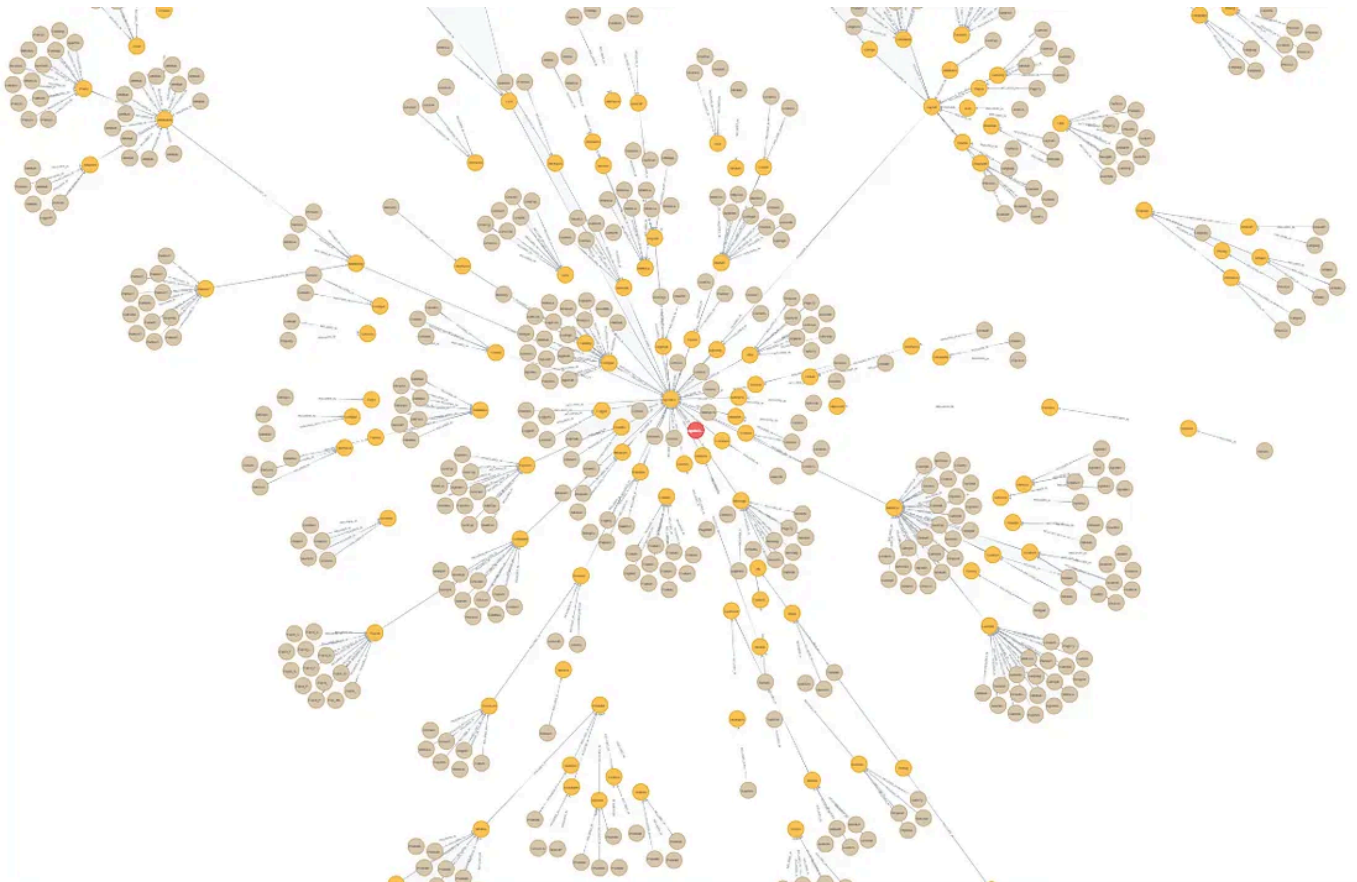
en.wikipedia.org

Exploration of files and folders is simple, as it returns the whole file system tree.

```
1 MATCH (p:Project { name: "ProjectName" })-[:INCLUDED_IN]->(root:Folder)<
2 RETURN p, root, fs
```

strazh-fs.cql hosted with ❤ by GitHub

[view raw](#)



Yellow nodes are Folders and grey nodes are Files. Red node is Project.

Types

Next layer is a collection of main code units: **classes** and **interfaces**.

Classes are an essential part of your codebase, and using this layer you can analyze OOP practices and design patterns.

Design Patterns - Wikipedia

Design Patterns: Elements of Reusable Object-Oriented Software (1994) is a software engineering book describing...

en.wikipedia.org

For example, let's take a look at each method that is declared in an interface and at the same time implemented in a class.

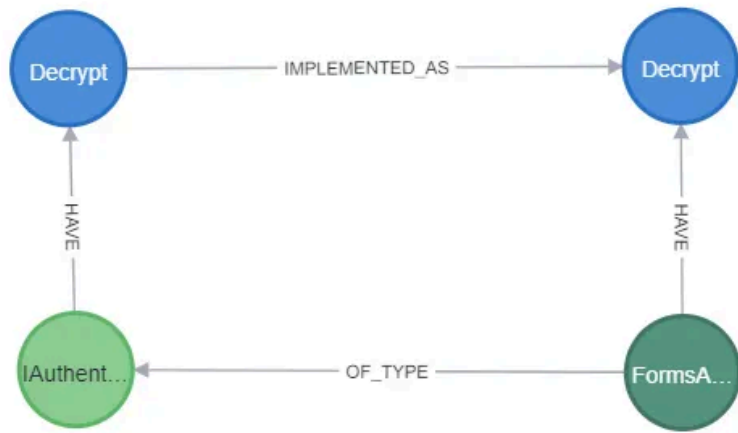
```
1 MATCH (impl:Method)<-[:HAVE]-(c:Class)-[:OF_TYPE]->(i:Interface)-[:HAVE]
2 WHERE impl.name = m.name
3 AND impl.arguments = m.arguments
4 AND impl.returnType = m.returnType
5 RETURN m.name as method, i.name as from_interface, c.name as impl_in_class
6 // MERGE (m)-[:IMPLEMENTED_AS]->(impl)
```

strazh-impl.cql hosted with ❤ by GitHub

[view raw](#)

We can build a new Relationship `[:IMPLEMENTED_AS]` and use it for more efficient traversing across the codebase. This also enriches our knowledge graph with a higher level concept to have a better language to query and explore the software design space.

Visually it is beautiful like this:



Methods

Methods **invoke** other methods and **instantiate** types.

This layer is a core of application business logic and builds a major part of the graph. Having this layer you can traverse in a program business logic and find relationships between application functionality.

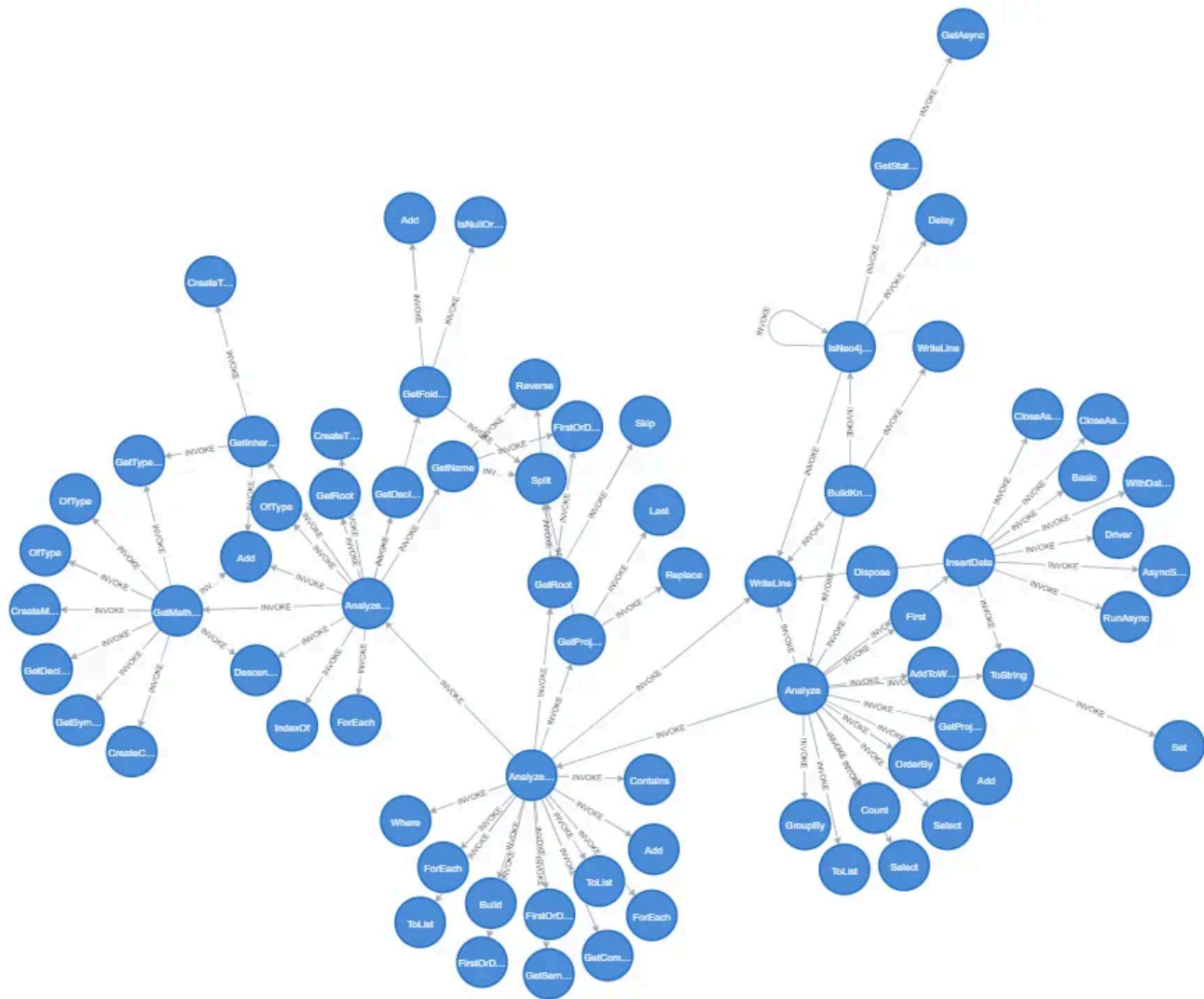
For instance, we can find out which other methods calls our method “BuildKnowledgeGraph” transitively.

```

1 MATCH path = (m:Method)-[:INVOKE*]->(:Method)
2 WHERE m.name = "BuildKnowledgeGraph"
3 RETURN path
  
```

strazh-invoke.cql hosted with ❤ by GitHub

[view raw](#)



Invocation chain of method in Strazh project started from method “BuildKnowledgeGraph”

Now let’s try to build a Codebase Knowledge Graph from your codebase!
Your codebase — is your Knowledge Graph.

Strazh in Action

Practical part

Strazh is a console app and uses a standard command line interface for execution.

Usage:

strazh [options]

Options:

-c, --credentials (REQUIRED)

required information in format `dbname:user:password` to connect to Neo4j Database

-t, --tier

optional flag as `project` or `code` or `all` (default `all`) selected tier to scan in a codebase

-d, --delete

optional flag as `true` or `false` or no flag (default `true`) to delete data in graph before execution

-s, --solution (REQUIRED)

optional absolute path to only one `.sln` file (can't be used together with -p / --projects)

-p, --projects (REQUIRED)

optional list of absolute path to one or many `.csproj` files (can't be used together with -s / --solution)

--version

show version information

-, -h, --help

show help and usage information

You can ask to analyze codebase based on single **solution** file or provide a list of any amount of **projects** — but not a mix of them. You also need to provide **credentials** for Neo4j Database and specify a **tier** to analyze. Strazh analyze and build a graph only for specified tiers.

Tier “Project” includes two graph layers:

- Project Dependencies
- Folder Structure

Tier “Code” includes two graph layers:

- **Types**
- **Methods**

The Tier separation was created specifically to differentiate our need of Architectural and Code level results. When you only need the Project diagram, run “Project” tier and save some time.

Docker

Strazh is available as [docker image in a Docker Hub](#). Simply pull and run a container. I recommend you use a docker-compose file to speed up the process.

Example 1: Solution

In this docker-compose file example we will analyze a Solution file with only a “Project” tier. This will result in a graph with only **projects** and their **nuget packages** and the dependencies between them.

```
1  version: '3'
2  services:
3
4    strazh:
5      image: vladbatushkov/strazh:latest
6      container_name: strazh
7      network_mode: host
8      volumes:
9        - {FULLPATH-TO-FOLDER}:/dest
10     environment:
11       - c=neo4j:neo4j:strazh
12       - t=project
13       - s=/dest/{SOLUTION-FILE}.sln
14     depends_on:
15       - neo4j
16
17     neo4j:
18       image: neo4j:4.2.0
19       container_name: strazh_neo4j
20       restart: unless-stopped
21       ports:
22         - 7474:7474
23         - 7687:7687
24       environment:
25         NEO4J_AUTH: neo4j/strazh
26         NEO4J_dbms_memory_pagecache_size: 1G
27         NEO4J_dbms_memory_heap_initial_size: 1G
28         NEO4J_dbms_memory_heap_max_size: 1G
29         NEO4JLABS_PLUGINS: "[\"apoc\", \"graph-data-science\"]"
30
```

strazh-docker-compose.yml hosted with ❤ by GitHub

[view raw](#)

Several tips about the docker:

- You need a running instance of Neo4j Database and grant to Strazh a role with write permissions. Run services one after another.
- Easiest setup is to use a default database “neo4j” with a default admin role “neo4j.” As you can see, provided “neo4j:neo4j:strazh” credentials

match prepared NEO4J_AUTH: neo4j/strazh settings in Neo4j.

- Also it's always good to have Neo4j with APOC and GDS plugins inside to power up your queries.

Example 2: Projects

Another example shows how to build a Codebase Knowledge Graph with default “All” tier for two explicitly provided projects. This example perfectly matches the classic pair: Project + Unit-Tests for it.

```
1  version: '3'
2  services:
3
4    strazh:
5      image: vladbatushkov/strazh:latest
6      container_name: strazh
7      network_mode: host
8      volumes:
9        - {FULLPATH-TO-ROOT-FOLDER}:/dest
10     environment:
11       - c=neo4j:neo4j:strazh
12       - p=/dest/{PATH-TO-PROJECT-1}/{PROJECT-FILE-1}.csproj /dest/{PATH-TO-PROJECT-1}
13     depends_on:
14       - neo4j
15
16   neo4j:
17     image: neo4j:4.2.0
18     container_name: strazh_neo4j
19     restart: unless-stopped
20     ports:
21       - 7474:7474
22       - 7687:7687
23     environment:
24       NEO4J_AUTH: neo4j/strazh
25       NEO4J_dbms_memory_pagecache_size: 1G
26       NEO4J_dbms_memory_heap_initial_size: 1G
27       NEO4J_dbms_memory_heap_max_size: 1G
28       NEO4JLABS_PLUGINS: ["apoc","graph-data-science"]
29
```

strazh-2-docker-compose.yml hosted with ❤ by GitHub

[view raw](#)

The Neo4j service configuration is the same. If you set up everything correctly you should see in the output something like this:

```
strazh | Neo4j is ready to use.
strazh | Brewing a Code Knowledge Graph of tier "All".
strazh | Setup analyzer...
strazh | Analyzer ready to analyze 2 project/s.
strazh | Project #1:
strazh | Analyzing Strazh.Tests.ProjectB project...
strazh | Analyzing Project tier...
strazh | Analyzing Project tier complete.
strazh | Analyzing Code tier...
strazh | Analyzing Code tier complete.
strazh | Analyzing Strazh.Tests.ProjectB project complete.
strazh | Code Knowledge Graph use "neo4j" Neo4j database.
strazh | Deleting graph data of "neo4j" database...
strazh | Deleting graph data of "neo4j" database complete.
strazh | Processing 31 triples...
strazh | Processing 31 triples complete.
strazh | Project #2:
strazh | Analyzing Strazh.Tests.ProjectA project...
strazh | Analyzing Project tier...
strazh | Analyzing Project tier complete.
strazh | Analyzing Code tier...
strazh | Analyzing Code tier complete.
strazh | Analyzing Strazh.Tests.ProjectA project complete.
strazh | Code Knowledge Graph use "neo4j" Neo4j database.
strazh | Processing 9 triples...
strazh | Processing 9 triples complete.
strazh | Code Knowledge Graph created.
strazh | exited with code 0
```

Strazh output

Now you can open a Neo4j Browser (<https://localhost:7474>) and start your first graph-based code analysis!



I hope that Strazh will help you in your project someday. If so, please don't forget to share your case in the comments.

Thanks for reading.

Looking for Contributors

I wrote a first line of code for a Strazh project in March 2020, and finally it's an open-source project with the early *beta* release in June 2021.

I am a novice in the open-source world, and a lot of things in the Strazh project need to be improved.

This is why I need your help! If you're interested in an exotic project like Strazh, please fork it and contribute.

vladbatushkov/strazh

Your codebase - is your Knowledge Graph. Strazh is a .NET Core console app to build a Codebase Knowledge Graph from a...

github.com

Contributors are welcome!

**Personal Thank you**

Peter Szabo

Sushmi Shrestha

Sergey Meshcheryakov

Royee Goldberg

Keattisak Chinburarat

References

- <https://threat.tevora.com/visualizing-dotnet-class-relationships-neo4j/>
- <https://github.com/daveaglick/Buildalyzer>

Software Development

Csharp

Neo4j

Knowledge Graph

Open Source



Published in Neo4j Developer Blog

Follow

13.4K followers · Last published 6 days ago

Developer content about graph databases, Neo4j, Cypher, data science, graph analytics, GraphQL, and more



Written by Vlad Batushkov

Follow

342 followers · 33 following

Engineering Manager @ Agoda. Neo4j Ninja. Articles brewed on modern tech, hops and indie rock'n'roll.

